Министерство образования и науки Российской Федерации
Санкт-Петербургский государственный политехнический университет
—
Институт компьютерных наук и кибербезопасности
Высшая школа кибербезопасности

НАУЧНО-ИССЛЕДОВАТЕЛЬСКАЯ РАБОТА
«Угрозы информационной безопасности, связанные с использованием микроконтроллеров с USB-интерфейсом»

Выполнил
студент гр. 5351001/20502 Улановский Г.А.
<подпись>

Преподаватель Макаров А.С.
<подпись>

Санкт-Петербург
2026

СОДЕРЖАНИЕ

Введение **3**

ГЛАВА 1. Микроконтроллеры с USB-интерфейсом и концепция BadUSB (обзор) **7**

1.1. Концепция атак типа BadUSB. 7

1.2. USB-микроконтроллеры для эмуляции HID: Digispark и аналоги. 7

ГЛАВА 2. Уязвимости интерфейса HID и классификация атак BadUSB 9

2.1. Упрощённая модель угроз BadUSB. 9

2.2. Анализ уязвимостей USB/HID-интерфейса. 11

2.3. Классификация атак с использованием USB-микроконтроллеров..... 13

ГЛАВА 3. Экспериментальное исследование USB/HID-устройств и элементов двухуровневой методики **17**

3.1. Методика и структура экспериментального стенда. 17

3.2. Формирование исходного корпуса HID-дескрипторов..... 18

3.3. Реализация mutator, ориентированного на структуру дескрипторов, и генерация вариантов. 19

3.4. Практическая проверка механизмов защиты на Linux-стенде..... 20

3.5. Аппаратная реализация второго уровня на RP2040 Zero. 21

3.6. Результаты экспериментов и их интерпретация. 21

ГЛАВА 4. Меры защиты от атак типа BadUSB **24**

4.1. Организационные меры и политика безопасности. 24

4.2. Программно-технические методы защиты..... 26

Заключение **31**

Список использованных источников **33**

ВВЕДЕНИЕ

В последние годы выявлен особый класс атак BadUSB, при котором внешнее USB-устройство ведёт себя не так, как ожидает пользователь или операционная система[1]. Злоумышленники используют возможность перепрограммирования прошивки USB-микроконтроллеров, чтобы маскировать устройство под доверенное (например, под клавиатуру или сетевую карту) и автоматически выполнять вредоносные действия[1]. Такая атака действует на уровне прошивки устройства, из-за чего антивирусы и обновления драйверов не могут устранить эту уязвимость[2]. Система распознаёт поддельное устройство по заявленному типу и доверяет ему, не имея средств проверки его истинных функций[3]. В результате вредоносный код может выполнить свои задачи до вмешательства средств защиты, причём пользователь не заметит подозрительной активности[3]. Подобный подход позволяет обходить привычные механизмы защиты и внедрять код даже без сетевого подключения[2], что делает BadUSB-атаки серьёзной угрозой для информационной безопасности.

Актуальность исследования обусловлена широким распространением USB-устройств и отсутствием встроенных механизмов аутентификации микропрошивок в протоколе USB. Обычная на вид флешка или клавиатура может оказаться инструментом атаки, способным скомпрометировать систему[2]. Особенно опасны миниатюрные микроконтроллерные платы (такие как Digispark), которые могут быть незаметно использованы злоумышленниками. Исследование направлено на выявление и анализ угроз, связанных с устройствами класса BadUSB, а также на разработку и апробацию элементов двухуровневой методики исследования USB/HID-угроз, сочетающей анализ на стороне хоста и аппаратное воспроизведение сценариев на микроконтроллерной платформе.

Цель исследования: выявить и проанализировать угрозы

информационной безопасности, связанные с использованием микроконтроллеров с USB-интерфейсом в сценариях атак класса BadUSB, а также разработать и апробировать элементы двухуровневой методики исследования USB/HID-угроз, включающей анализ на стороне хоста и мутацию HID-дескрипторов и аппаратное воспроизведение сценариев на микроконтроллерной платформе.

Для достижения поставленной цели необходимо решить следующие задачи:

а) Изучить архитектуру и возможности микроконтроллеров с USB-интерфейсом, применяемых в атаках класса BadUSB, включая платы Digispark, RP2040 Zero и аналогичные устройства, способные эмулировать USB/HID-периферию.

б) Проанализировать уязвимости протокола USB и интерфейса HID, на которых основываются атаки BadUSB, и установить, почему современные операционные системы доверяют устройствам ввода без проверки происхождения и подлинности.

в) Классифицировать возможные атаки с использованием USB-микроконтроллеров и сформировать упрощённую модель угроз для атак данного класса.

г) Сформировать экспериментальный стенд для исследования USB/HID-устройств, включающий сбор USB-трасс, извлечение и анализ реальных отчетов о HID-дескрипторах, а также подготовку исходного корпуса дескрипторов.

д) Реализовать прототип mutator, ориентированного на структуру дескрипторов для генерации модифицированных вариантов HID-дескрипторов и оценить пригодность такого подхода для первого уровня методики.

е) Исследовать практические механизмы защиты от атак класса BadUSB на Linux-стенде, включая штатную авторизацию USB-устройств и политику фильтрации устройств с использованием USBGuard.

ж) Реализовать аппаратный этап методики на базе микроконтроллерной платформы RP2040 Zero и подтвердить возможность воспроизведения USB/HID-сценариев на реальном устройстве.

Объект исследования: информационные системы общего назначения и их подсистемы обработки USB/HID-устройств, подвергающиеся воздействию вредоносных или модифицированных USB-устройств.

Предмет исследования: методы осуществления и выявления несанкционированных воздействий на информационные системы посредством микроконтроллеров с USB-интерфейсом, а также методы ограничения и фильтрации таких воздействий на уровне операционной системы и политики подключения устройств.

Методы исследования: анализ литературы и открытых источников по теме BadUSB и USB/HID-угроз; метод сравнительного анализа и классификации для систематизации видов атак; моделирование сценариев угроз; экспериментальный метод, включающий сбор USB-трасс, извлечение и анализ реальных HID-дескрипторов, генерацию модифицированных вариантов дескрипторов, исследование механизмов Linux authorization и USBGuard, а также аппаратное воспроизведение USB/HID-сценариев на платформе RP2040 Zero.

Научная новизна: работа систематизирует сведения об атаках класса BadUSB и переносит акцент с демонстрации отдельных HID-пейлоадов на двухуровневый подход к исследованию USB/HID-угроз. В работе предложено и частично реализовано сочетание анализа на стороне хоста и мутации реальных HID-дескрипторов с последующим аппаратным воспроизведением USB/HID-сценариев на микроконтроллерной платформе. Полученные

результаты показывают, что исследование USB/HID-угроз может строиться не только как демонстрация атакующего скрипта, но и как последовательная методика анализа, формирования сценариев и аппаратного подтверждения их реализуемости.

Практическая значимость: результаты исследования могут быть использованы для повышения защищённости рабочих мест и исследовательских стендов от атак класса BadUSB. Сформированный корпус HID-дескрипторов, реализованный прототип mutator, ориентированный на структуру дескрипторов, результаты проверки Linux authorization и USBGuard, а также аппаратный PoC на RP2040 Zero могут применяться при разработке локальных политик безопасности, выборе механизмов device control и подготовке специалистов в области кибербезопасности.

ГЛАВА 1. МИКРОКОНТРОЛЛЕРЫ С USB-ИНТЕРФЕЙСОМ И КОНЦЕПЦИЯ BADUSB (ОБЗОР)

1.1. Концепция атак типа BadUSB.

В этой части даётся общее представление о природе атак BadUSB. Описана история появления термина (исследование Карстена Нолла и Якоба Лелла, Black Hat USA 2014) и суть уязвимости USB-протокола – отсутствие механизмов проверки прошивки устройств[4][5]. Приводятся примеры, как USB-устройства с перепрошиваемым микроконтроллером могут выдавать себя за устройства другого типа (флешка как клавиатура, сетевой адаптер и т.д.) и выполнять неожиданные для пользователя функции[1]. Отмечается, что BadUSB-атаки не эксплуатируют программные уязвимости ОС, а используют фундаментальные свойства аппаратного интерфейса USB, из-за чего стандартными методами обнаружить их крайне сложно[6]. Также рассмотрены типовые сценарии использования BadUSB (целевое внедрение устройства-жертвы, социальная инженерия с «забытыми» носителями, атаки через цепочку поставок) для иллюстрации масштабов угрозы.

1.2. USB-микроконтроллеры для эмуляции HID: Digispark и аналоги.

Здесь рассматриваются аппаратные средства, применяемые злоумышленниками в подобных атаках. Приводится обзор устройств класса **BadUSB**: от перепрошитых *USB-флешек* до специализированных плат-разработчиков. Особое внимание уделяется категории **псевдо-USB-устройств** – миниатюрных микроконтроллерных плат (Arduino Leonardo/Pro Micro, Teensy, Rubber Ducky, Flipper Zero и др.), предназначенных для имитации HID-периферии[7]. На примере платы **Digispark Attiny85** показывается, что такие устройства очень компактны и доступны по цене (несколько долларов США[8]), что повышает риск их широкого применения злоумышленниками. Описаны технические возможности Digispark: использование V-USB для

эмуляции USB-интерфейса, поддержка библиотеки DigiKeyboard (эмуляция клавиатуры), ограниченные ресурсы памяти (~6 КБ для полезной нагрузки) и отсутствие аппаратной поддержки высокоскоростного USB. Также приводятся сведения о сходных устройствах (например, использование Arduino Pro Micro как HID-эмулятора) и сравнительная характеристика их возможностей. В заключение этой главы подчёркивается, что доступность и простота программирования таких плат открывает возможности для проведения атак даже начинающим злоумышленником.

ГЛАВА 2. УЯЗВИМОСТИ ИНТЕРФЕЙСА HID И КЛАССИФИКАЦИЯ АТАК BADUSB

2.1. Упрощённая модель угроз BadUSB.

В данном пункте формализуется сценарий атаки BadUSB для последующего анализа и классификации. Модель описывает основные элементы:

– нарушитель – субъект, располагающий устройством на основе микроконтроллера (например, платой Digispark) и способный обеспечить физическое подключение к USB-порту целевой системы (путём непосредственного доступа, «подклада» устройства или социальной инженерии);

– объект атаки – персональный компьютер под управлением операционной системы Windows, на котором пользователь уже выполнил вход в систему и где действуют типичные политики безопасности по умолчанию;

– уязвимость – доверие операционной системы к устройствам ввода класса HID (клавиатура, мышь) без криптографической проверки их подлинности и без различения «человеческого» и автоматически сгенерированного ввода;

– цель атакующего – получение несанкционированного доступа к системе, выполнение произвольного кода, модификация параметров безопасности либо нарушение конфиденциальности и/или целостности обрабатываемых данных.

Сценарий развития угрозы в базовом виде можно описать следующей последовательностью шагов:

1. Нарушитель обеспечивает подключение устройства BadUSB к USB-порту целевого ПК (непосредственно либо за счёт действий пользователя).

з) Операционная система идентифицирует устройство как HID-клавиатуру (или комбинацию HID-устройств) и автоматически активирует драйвер ввода.

и) После короткой задержки микроконтроллер начинает автоматизированный ввод запрограммированной последовательности клавиш: открытие окна «Выполнить» или терминала, запуск cmd/PowerShell, ввод команд для загрузки и запуска полезной нагрузки и т.п.

к) На стороне ОС выполняются команды с правами текущего пользователя; при наличии достаточных прав возможна эскалация привилегий, изменение политик безопасности, создание скрытой учётной записи, развёртывание постоянного бэкдора и др.

л) После завершения сценария устройство может завершить активность или перейти к пассивному режиму, внешне оставаясь обычной «клавиатурой».

В модели фиксируются ключевые **допущения**:

– система разблокирована пользователем (сеанс активен);

– у текущего пользователя имеются права, достаточные для выполнения критичных операций (например, добавление локального администратора, изменение настроек брандмауэра);

– средства защиты не блокируют автоматически запуск командной строки/PowerShell и типичные вспомогательные утилиты.

Также учитываются **ограничения**: скорость «печати» с платы Digispark и необходимость имитации естественных задержек между нажатиями, возможная реакция антивируса/EDR на типичные цепочки команд, влияние механизмов UAC и политик групп на успешность выполнения сценария.

Такая упрощённая модель угроз позволяет чётко очертить границы

исследования: кто и при каких условиях может реализовать атаку, какие ресурсы затрагиваются и какие предпосылки необходимы. В дальнейшем она служит основой для детализации уязвимостей USB/HID-интерфейса и прикладочной классификации атак BadUSB.

2.2. Анализ уязвимостей USB/HID-интерфейса.

В этом пункте подробно рассматриваются причины, по которым атаки типа BadUSB в принципе возможны и практически реализуемы. Ключевую роль играет архитектура стандарта USB и используемая в современных ОС модель доверия к устройствам ввода.

Во-первых, в актуальных версиях стандарта USB не предусмотрена обязательная криптографическая аутентификация прошивки периферийных устройств. Контроллер хоста опирается на описатели устройства (device descriptors), передаваемые самим устройством: оно самостоятельно заявляет, к какому классу относится (например, HID-клавиатура или HID-мышь), какие интерфейсы поддерживает и т.д. Это означает, что модифицированная прошивка имеет возможность выдавать себя за заведомо доверенный тип оборудования[5][3].

Во-вторых, в типичных конфигурациях операционные системы автоматически доверяют устройствам класса HID, так как без них невозможна штатная работа пользователя. После подключения новой USB-клавиатуры драйвер HID загружается без дополнительных вопросов, а все поступающие от неё сообщения рассматриваются как легитимные действия пользователя. Системе, как правило, не из чего сделать вывод, что «клавиатура» управляется не человеком, а микроконтроллером.

На практике это означает, что вредоносное HID-устройство может немедленно после инициализации запустить заранее подготовленный сценарий: открыть окно «Выполнить» или терминал, ввести команды для

загрузки и исполнения скрипта, изменить параметры системы и т.п., причём всё это будет выглядеть как серия обычных действий пользователя[3]. При достаточно коротком сценарии атака может завершиться раньше, чем пользователь успеет заметить что-то необычное или чем средства защиты среагируют на побочные артефакты (запуск PowerShell, сетевую активность и пр.).

Дополнительным фактором риска является возможность эмуляции различных классов устройств одним и тем же микроконтроллером. В частности, вместо (или помимо) HID-клавиатуры USB-устройство может объявить себя сетевым адаптером (USB-Ethernet) и интегрироваться в сетевую конфигурацию хоста. В таком случае появляется возможность перенаправлять трафик через узел, контролируемый злоумышленником, реализуя атаки типа «человек посередине» (MITM), подмену DNS и другие сетевые атаки[9].

В анализ также включаются технологические ограничения и особенности реализации:

– необходимость подбора реалистичной скорости ввода и задержек между командами, чтобы не провоцировать подозрительное поведение интерфейса;

– влияние механизмов контроля учётных записей (UAC) в Windows: необходимость запуска консоли с повышенными правами, возможные обходные пути;

– работа антивирусов и EDR-систем, способных по поведенческим признакам распознать типичные цепочки команд, используемые в BadUSB-пейлоадах.

По результатам анализа можно выделить несколько критических уязвимых точек:

– безусловное доверие к HID-устройствам ввода без проверки происхождения и подлинности;

– отсутствие механизма разграничения «человеческого» и автоматически сгенерированного ввода;

– возможность эмуляции разных типов устройств (HID, сетевой адаптер, накопитель) на базе одного микроконтроллера;

– недостаточная интеграция средств защиты с подсистемой ввода (антивирус видит уже последствия, но не сам факт подозрительной автоматизации ввода).

Эти особенности создают благоприятные условия для реализации атак BadUSB и задают рамки для последующей классификации атакующих сценариев.

2.3. Классификация атак с использованием USB-микроконтроллеров.

На основе разобранных уязвимостей предлагается прикладная классификация атак BadUSB, реализуемых на базе USB-микроконтроллеров. В первую очередь учитываются **тип эмулируемого устройства** и **характер воздействия на целевую систему**.

Эмуляция HID-клавиатуры:

Это наиболее распространённый и практически значимый класс. Устройство автоматически вводит запрограммированную последовательность нажатий клавиш и тем самым реализует сценарий атаки. Возможные цели включают:

– запуск cmd/PowerShell и выполнение вредоносных командных скриптов (загрузка и запуск исполняемых файлов, скриптов, бэкдоров и т.п.);

– создание локального администратора с последующим входом под этой учётной записью (при наличии достаточных прав у текущего пользователя);

– изменение параметров системы безопасности: отключение брандмауэра, изменение политик SmartScreen, добавление исключений в антивирус и др.;

– внедрение в механизмы автозагрузки (регистровые ключи, папки автозапуска, планировщик заданий) для обеспечения постоянного присутствия вредоносного кода;

– запуск вспомогательных инструментов (кейлоггеров, утилит удалённого администрирования и др.), если они уже присутствуют в системе или могут быть быстро загружены;

– вывод ложных окон и сообщений пользователю (элемент социальной инженерии), имитирующих системные диалоги, с целью заставить его ввести учетные данные или выполнить дополнительные действия.

Характерная особенность этого класса атак — минимальное участие пользователя после факта подключения устройства: все дальнейшие шаги выполняются сценарно и с высокой скоростью. Например, устройство может ввести последовательность команд, которая скачивает и запускает исполняемый файл, при этом сам файл может храниться лишь в памяти и не оставлять на диске очевидных артефактов[10].

Эмуляция сетевого адаптера:

В данном случае устройство выдаёт себя за USB-Ethernet или аналогичный сетевой интерфейс. После подключения операционная система может автоматически добавить новый интерфейс и частично маршрутизировать через него трафик. Такой сценарий позволяет:

– перенаправить сетевой трафик жертвы через контролируемый злоумышленником узел;

– перехватывать и модифицировать передаваемые данные (включая учётные данные);

– подменять параметры сетевой конфигурации (DNS-сервер, шлюз по умолчанию) для обхода фильтрации[11].

Данный класс атак редко реализуется на платах уровня Digispark из-за ограниченных ресурсов контроллера, но практически реализуем на более продвинутых платформах и специализированных устройствах. В контексте данной работы он рассматривается как сопутствующий, основное внимание уделяется HID-сценариям.

Эмуляция загрузочного устройства (Boot Device):

Более сложный вариант, при котором USB-устройство имитирует флеш-накопитель или оптический диск с загрузочным сектором. Целью выступает компрометация системы на этапе загрузки ОС. Потенциальные векторы:

– изменение порядка загрузки в BIOS/UEFI с принудительным запуском с внешнего носителя;

– подмена загрузочных записей или внедрение в начальные стадии загрузчика ОС;

– установка бэкдора на уровне kernel или драйверов ещё до старта пользовательского пространства[12].

Такие атаки требуют отдельной подготовки (влияния на конфигурацию BIOS/UEFI, наличия контроля над физическим доступом к машине на момент перезапуска) и в рамках данной НИР подробно не реализуются, но упоминаются как теоретически возможное развитие BadUSB-подхода.

Комбинированные атаки:

В этом классе устройства используют сразу несколько профилей: например, одновременно эмулируют HID-клавиатуру и накопитель, либо комбинируют HID и сетевой адаптер. Возможные сценарии:

– первоначально устройство выступает как HID-клавиатура, выполняет автоматизированный ввод команд для загрузки/активации

вредоносного ПО, а затем «открывает» накопитель для обмена файлами или скрытого копирования данных;

– одновременная эмуляция клавиатуры и мыши для обхода ограничений, связанных только с клавиатурным вводом (например, подтверждение определённых действий кликом мыши);

– использование HID-скрипта для незаметного изменения сетевых настроек, после чего задействуется эмулируемый сетевой интерфейс.

Подобные сценарии сложнее в реализации с точки зрения прошивки и тестирования, но демонстрируют потенциал развития BadUSB-устройств и важность комплексного подхода к защите.

Предлагаемая классификация носит прикладной характер: она связывает технические возможности USB-микроконтроллеров с целями атакующего и последствиями для системы. На её основе в последующих главах формулируются требования к мерам защиты: от противодействия автоматическому вводу команд и ограничения доверия к новым HID-устройствам до контроля сетевых подключений и процедур безопасной обработки USB-носителей.

ГЛАВА 3. ЭКСПЕРИМЕНТАЛЬНОЕ ИССЛЕДОВАНИЕ USB/HID-УСТРОЙСТВ И ЭЛЕМЕНТОВ ДВУХУРОВНЕВОЙ МЕТОДИКИ

3.1. Методика и структура экспериментального стенда.

В рамках практической части работы был развёрнут экспериментальный стенд, предназначенный для исследования USB/HID-устройств, формирования исходного корпуса дескрипторов, проверки механизмов защиты и подтверждения возможности аппаратного воспроизведения сценариев на микроконтроллерной платформе.

Стенд был построен на базе системы Ubuntu Linux. Для регистрации USB-обмена применялся штатный механизм трассировки usbmon, позволяющий фиксировать события перечисления устройств, управляющие запросы и обмен по конечным точкам. Для анализа HID-подсистемы использовались системный интерфейс /sys/bus/hid/devices/*/report_descriptor, а также утилита usbhid-dump, позволяющая получить реальные отчеты о HID-дескрипторах подключённых устройств.

Методика эксперимента строилась в два уровня. Первый уровень был ориентирован на исследование на стороне хоста: извлечение реальных HID-дескрипторов, формирование исходного корпуса, реализацию mutator, ориентированного на структуру дескрипторов и анализ реакции хоста на подключаемые устройства и защитные механизмы. Второй уровень был ориентирован на аппаратное подтверждение реализуемости подхода: в роли USB/HID-устройства использовалась плата RP2040 Zero с стеком USB устройства на базе TinyUSB.

Таким образом, практическая часть была организована не как единичная демонстрация заранее подготовленной нагрузки, а как последовательное исследование: сбор реальных десрипторов, генерация модифицированных вариантов, проверка защитных механизмов и перенос части сценариев на аппаратный уровень.

3.2. Формирование исходного корпуса HID-дескрипторов.

На первом этапе был сформирован исходный корпус HID-Descriptor сообщений, используемый как база для дальнейшего анализа и генерации модифицированных вариантов. В качестве источника были выбраны реально подключённые к Linux-хосту HID-устройства: USB-мышь Acer Elite USB Optical Mouse и USB-клавиатура Acer Elite USB Keyboard, для которой отдельно учитывались два HID-интерфейса.

В результате были выделены три исходных дескриптора:

1. дескриптор USB-мыши;

м) дескриптор основного интерфейса клавиатуры;

н) дескриптор дополнительного consumer/media-интерфейса клавиатуры.

Каждый из дескрипторов был сохранён в hex-представлении и включён в исходный корпус. Для исходных данных был выполнен первичный структурный анализ: длина дескрипторов, наличие и число тегов Report Size, Report Count, Report ID, количество Collection / End Collection, а также логические минимумы и максимумы. Такой корпус позволил перейти от абстрактного обсуждения HID-угроз к работе с реальными, используемыми в системе дескрипторов.

Практически это важно потому, что именно HID-report-descriptor описывает структуру будущих отчетов, размеры полей, типы данных и способ интерпретации ввода хостом. Следовательно, ошибки, аномальные значения или нетипичные сочетания полей в представлении на уровне дескрипторов могут служить источником проблем как для корректности работы устройства, так и для устойчивости стека обработки USB/HID.

3.3. Реализация mutator, ориентированного на структуру дескрипторов, и генерация вариантов.

На основе сформированного исходного корпуса был реализован прототип mutator, ориентированного на структуру дескрипторов. Его задача заключалась не в хаотическом побайтовом искажении исходных дескрипторов, а в ограниченных структурно осмысленных изменениях определённых полей. В качестве целевых параметров были выбраны:

- Report Size;
- Report Count;
- Logical Minimum;
- Logical Maximum;
- Report ID.

Такой выбор обусловлен тем, что именно эти элементы определяют размерность и интерпретацию будущих HID reports и потому представляют интерес как потенциальные источники нестандартного поведения хоста.

При генерации вариантов сохранялась общая структура дескриптора: не нарушался баланс Collection / End Collection, не разрушался общий формат HID-описания, а длина дескриптора внутри одной seed-группы оставалась неизменной. В результате было сгенерировано 30 мутантов — по 10 вариантов для каждого из трёх исходных дескрипторов.

Для каждого варианта автоматически фиксировались:

- длина дескриптора;
- число основных HID-тегов;
- принадлежность к seed-группе;
- журнал произведённых изменений.

Результаты были собраны в сводную CSV-таблицу. Таким образом, уже на первом уровне был реализован не единичный тест, а небольшой экспериментальный корпус вариантов на уровне дескрипторов, пригодный для

дальнейшего анализа и последующего переноса части сценариев на аппаратную платформу.

3.4. Практическая проверка механизмов защиты на Linux-стенде.

Отдельный этап практической части был посвящён исследованию встроенных и policy-based механизмов защиты от нежелательных USB-устройств на Linux-стенде.

Во-первых, был проверен штатный механизм авторизации USB-устройств на уровне ядра Linux. С использованием параметра authorized_default для выбранной USB-шины был включён режим запрета новых подключений по умолчанию. После этого тестовое USB-устройство (флеш-накопитель) определялось системой, но помечалось как неавторизованное и не допускалось к полноценному использованию. Далее вручную был выполнен перевод устройства в состояние authorized=1, после чего оно стало доступным системе. Эксперимент подтвердил практическую применимость данного механизма как простого способа ограничения подключения новых USB-устройств.

Во-вторых, был развёрнут USBGuard. Для уже присутствующих устройств была автоматически сгенерирована стартовая политика, после чего демон был запущен в конфигурации, при которой имеющиеся устройства сохранялись в рабочем состоянии, а новые подключения обрабатывались политикой. В дальнейшем был проверен более жёсткий режим, при котором новые устройства переводились в состояние блокировки. Результаты фиксировались через вывод lsusb, системные журналы и audit-журнал USBGuard.

Дополнительно был подготовлен пример policy-фрагмента, ориентированного на противодействие устройствам с подозрительной комбинацией интерфейсов, например mass storage + HID. Такой подход важен

для противодействия более сложным вариантам BadUSB, где устройство не ограничивается одной HID-ролью, а использует composite-поведение.

Таким образом, практическая часть работы включила не только исследование атакующего потенциала USB-устройств, но и проверку реальных защитных механизмов, способных ограничивать или блокировать подобные сценарии.

3.5. Аппаратная реализация второго уровня на RP2040 Zero.

Для подтверждения реализуемости аппаратного этапа методики была использована плата RP2040 Zero. На базе Pico SDK и примеров TinyUSB для RP2040 был собран и прошит USB HID device example, после чего Linux-хост корректно определил плату как HID-устройство. Факт перечисления был зафиксирован средствами lsusb, dmesg и usbmon.

На следующем этапе в исходный файл USB-дескрипторов TinyUSB-примера были внесены контролируемые изменения: изменён идентификатор продукта и строковый дескриптор устройства. В результате был получен модифицированный аппаратный вариант RP2040 HID Variant 1, который также успешно перечислялся в системе и определялся как HID-устройство. Для этого варианта была снята отдельная USB-трасса.

Данный этап принципиально важен для новой постановки темы, поскольку показывает, что результаты можно переносить на реальное USB-устройство. Иными словами, исследование не ограничивается анализом файлов дескрипторов на хосте: была подтверждена возможность воспроизвести USB/HID-сценарий на реальной микроконтроллерной платформе и наблюдать его перечисление и поведение в системе.

3.6. Результаты экспериментов и их интерпретация.

По итогам практической части были получены следующие результаты:

Во-первых, сформирован исходный корпус из трёх реальных HID-report-descriptors, соответствующих реально используемым в системе устройствам: USB-мыши и двум HID-интерфейсам клавиатуры. Это позволило перейти от абстрактного описания BadUSB к работе с реальными структурами HID-дескрипторов.

Во-вторых, реализован mutator, ориентированный на структуру дескрипторов, сгенерировавший 30 модифицированных вариантов на основе исходных дескрипторов — по 10 вариантов для каждого исходного дескриптора. Полученные варианты были систематизированы и описаны в виде таблиц, что создаёт основу для последующих более глубоких экспериментов.

В-третьих, практически подтверждена применимость Linux authorization и USBGuard как средств противодействия нежелательным USB-устройствам. Показано, что как штатные средства ядра, так и policy-based механизмы способны блокировать новые подключения и снижать риск успешной реализации атак класса BadUSB.

В-четвёртых, реализован аппаратный этап на RP2040 Zero. Получены два аппаратных варианта HID-устройства: базовый и модифицированный. Их корректное перечисление подтверждено в системе, а USB-трассы сохранены как экспериментальные артефакты.

В совокупности это позволяет сделать вывод, что на текущем этапе работы уже реализованы ключевые элементы двухуровневого подхода:

– Подготовка хоста и анализ дескрипторов;
– генерация модифицированных вариантов;
– проверка средств защиты;
– аппаратное подтверждение возможности воспроизведения USB/HID-сценариев.

При этом следует отметить, что первый уровень пока реализован как прототип descriptor-aware исследования, а не как полноценный coverage-guided fuzzing-фреймворк уровня USBFuzz. Тем не менее, даже в таком виде он уже формирует исследовательскую основу для дальнейшего расширения дипломной работы

ГЛАВА 4. МЕРЫ ЗАЩИТЫ ОТ АТАК ТИПА BADUSB

4.1. Организационные меры и политика безопасности.

Организационные меры являются первым и во многом определяющим рубежом защиты от атак типа BadUSB. Поскольку уязвимость заложена в самой архитектуре USB и модели доверия к устройствам ввода, задача организации состоит в том, чтобы минимизировать вероятность физического подключения вредоносного устройства и ограничить последствия такого подключения.

Во-первых, целесообразно формализовать в локальных нормативных актах политику использования USB-устройств. Такая политика должна включать:

– запрет на подключение к рабочим станциям личных, найденных или присланных по почте USB-устройств (флешек, кабелей, гаджетов и т.п.);

– закрепление перечня доверенных типов устройств и моделей (клавиатуры, мыши, сканеры штрих-кодов и др.), а также порядка их выдачи и замены;

– требование сдавать все подозрительные или найденные носители в подразделение ИБ для проверки, а не подключать их самостоятельно;

– правила использования публичных зарядных станций и чужих кабелей (рекомендация использовать только фирменные зарядные устройства либо адаптеры-«data blocker», отключающие линии передачи данных) [3].

Во-вторых, необходимо реализовать на практике принцип минимальных привилегий. Большинство сценариев BadUSB критичны именно потому, что выполняются с правами пользователя, обладающего административными полномочиями. Поэтому рекомендуется:

– по возможности лишить обычных пользователей локальных прав администратора;

– запретить запуск высокорисковых инструментов (PowerShell, cmd, wmic и т.п.) пользователями без необходимости либо требовать отдельного подтверждения администратора [3];

– ограничить установку новых драйверов и устройств: автоустановка новых HID-устройств должна быть либо запрещена, либо требовать одобрения администратора.

В-третьих, важнейшим элементом защиты является обучение и повышение осведомлённости персонала. Практика и полевые исследования показывают, что значительная доля людей автоматически подключает найденные USB-носители к своим ПК, не задумываясь о рисках [3]. В рамках программ информационной безопасности следует:

– отдельно разъяснять природу атак BadUSB и демонстрировать реальные примеры (например, автоматический запуск команд при подключении «безобидной» флешки);

– рассказывать про приёмы социальной инженерии, такие как USB baiting (раскидывание «забытых» флешек или рассылка носителей в виде «подарков»), и закреплять правило: любой неизвестный USB-носитель потенциально опасен;

– проводить периодические тренинги и, при необходимости, контролируемые испытания (например, разбрасывание учебных «ловушечных» флешек с последующим анализом, кто попытался их использовать) [3], [6].

Наконец, организационные меры должны предусматривать и контроль физического доступа к оборудованию. Для критичных рабочих мест и серверов рекомендуется:

– ограничить круг лиц, имеющих доступ к системным блокам и USB-портам;

– использовать физические заглушки и замки для неиспользуемых портов;

– по возможности выносить доступные пользователю порты в отдельные, контролируемые устройства (док-станции, терминалы), а порты на самом системном блоке закрывать.

В совокупности такие меры позволяют значительно снизить вероятность успешной реализации атак BadUSB за счёт уменьшения числа точек подключения и числа пользователей, готовых бездумно доверять неизвестным USB-устройствам.

4.2. Программно-технические методы защиты.

Технические меры направлены на то, чтобы обнаружить, заблокировать или ограничить вредоносную активность USB-устройств на уровне операционной системы, средств защиты и аппаратной инфраструктуры. В силу универсальности и низкоуровневости USB ни одна отдельная мера не даёт абсолютной защиты, поэтому важно сочетать несколько подходов.

Контроль устройств и белые списки (Device Control).

Современные ОС и решения класса Endpoint / DLP поддерживают механизмы фильтрации USB-устройств по атрибутам (классу, VID/PID, серийному номеру и др.). В Linux-среде примером такого подхода является фреймворк USBGuard, позволяющий задать политику: какие устройства разрешены, какие заблокированы, а какие требуют ручного одобрения [13]. Аналогичная логика реализуется в Windows через групповые политики и функции device control в корпоративных продуктах (Ivanti, DeviceLock и др.) [3].

Практически это выглядит так:

– на этапе инвентаризации формируется список доверенных USB-устройств (например, конкретные модели клавиатур и мышей, используемых в организации);

– политика переводится в режим «разрешить только известные устройства», все прочие автоматически блокируются или переводятся в режим «только зарядка/только чтение»;

– добавление нового устройства в доверенный список осуществляется централизованно ИБ-подразделением.

Такая модель существенно затрудняет использование произвольных микроконтроллерных плат в роли HID-эмуляторов: даже если устройство маскируется под клавиатуру, его идентификаторы не совпадут с доверенными, и ОС заблокирует его работу [11], [3]. Ограничение этого подхода — необходимость сопровождения списков (учёт замены периферии, новых моделей и т.д.).

Специализированные механизмы защиты от BadUSB в антивирусах и EDR.

Ряд современных средств защиты реализует отдельные компоненты для противодействия атакам класса BadUSB. Например, в корпоративных продуктах Kaspersky Endpoint Security реализован механизм авторизации USB-клавиатур: при подключении нового HID-устройства система выводит на экран код, который пользователь должен ввести именно с этой клавиатуры. Пока код не введён верно, устройство не считается доверенным и его ввод блокируется [11]. Очевидно, что типичный скрипт на Digispark или Rubber Ducky не сможет корректно пройти такую проверку.

Аналогичные функции реализуются и другими вендорами в рамках модулей Device Control и Endpoint Protection: можно не только разрешать/запрещать устройства по идентификаторам, но и накладывать дополнительные условия (ручное подтверждение при первом подключении,

logging, запрос одобрения администратора и т.п.) [3]. Здесь важно именно включить и правильно настроить эти модули: по умолчанию во многих продуктах они находятся в «мягком» режиме или отключены.

Поведенческий мониторинг и детекция аномального ввода.

Даже если устройство оказалось допущено в систему, его активность может быть выявлена по поведенческим признакам. В частности, предлагаются следующие подходы:

– отслеживание супербыстрого ввода: программные детекторы (по типу утилит класса DuckHunt) анализируют скорость печати и блокируют устройства, которые вводят сложные команды со скоростью, недостижимой для человека [6];

– корреляция событий подключения HID-устройства и запуска высокорисковых процессов: если сразу после появления новой клавиатуры автоматически стартует cmd.exe или powershell.exe с нетипичными параметрами, это может рассматриваться как индикатор атаки Rubber Ducky / BadUSB [6];

– анализ последовательностей команд (частые обращения к утилитам администрирования, попытки изменения политик безопасности и т.п.) в сочетании с данными о недавних подключениях устройств.

Такие методы частично реализуются в современных EDR/EDP-решениях, а также могут быть интегрированы в SIEM: события о подключении USB-клавиатур и запуске критичных процессов передаются в систему корреляции и порождают оповещения при выявлении подозрительных шаблонов [6]. Ограничение заключается в том, что злоумышленник может замедлить ввод и сделать поведение более «похожим» на человека, а чрезмерно агрессивные пороги могут приводить к ложным срабатываниям.

Аппаратные средства защиты USB-портов.

Помимо программных механизмов используются и аппаратные

решения:

– физическая блокировка портов (замки, заглушки) на критичных рабочих станциях и серверах;

– использование data-blocker-адаптеров («USB-кондомов»), передающих только питание без линий данных, для безопасной зарядки мобильных устройств от чужих портов и станций [3];

– применение специализированных USB-фаерволов, таких как проект USG, который выступает в роли прокси между ПК и потенциально недоверенным USB-устройством, пропуская только строго ограниченный набор протокольных взаимодействий и отфильтровывая опасные комбинации (например, скрытую HID-функциональность внутри флешки) [15].

Подобные устройства особенно актуальны в лабораториях, на рабочих местах администраторов и в иных зонах повышенного риска, где по роду деятельности приходится подключать большое количество внешних носителей. Недостатком является стоимость и ограниченная совместимость: USB-фаерволы, как правило, поддерживают только распространённые классы устройств и скорости USB 2.0.

Дополнительно появляются узкоспециализированные средства, ориентированные на выявление поддельных кабелей и периферии. Например, детекторы вредоносных кабелей класса O.MG анализируют потребление тока и поведение USB-устройства и позволяют выявить встроенные микроконтроллеры и радиомодули, отсутствующие в обычных кабелях [16].

Перспективные направления: аутентификация и изоляция USB-устройств.

Стратегическим направлением развития защиты является внедрение криптографической аутентификации USB-устройств и проверка целостности их прошивок. В рамках стандарта USB Type-C уже предложен механизм USB Type-C Authentication, позволяющий хосту проверять сертификат устройства

до установления полноценного соединения [3]. Теоретически, это может препятствовать использованию несертифицированных микроконтроллеров в качестве HID-устройств.

На уровне исследовательских прототипов рассматриваются решения по типу SandUSB, которые изолируют подключаемые USB-устройства в отдельной программной «песочнице» и фильтруют их взаимодействие с основной системой [17], а также методы “USB-fingerprinting”, позволяющие по тонким различиям во временных характеристиках и структуре USB-трафика отличать легитимные устройства от подделок и выявлять смену прошивки [16]. Пока такие подходы не получили широкого промышленного внедрения, однако они обозначают возможный путь эволюции стандарта USB в сторону встроенных механизмов доверия.

В целом, программно-технические меры защиты от BadUSB-атак должны рассматриваться не изолированно, а в сочетании с организационными: даже самая совершенная фильтрация не защитит от устройства, которое администратор сам внёс в белый список, следуя просьбе злоумышленника, и наоборот, строгая политика без технической поддержки затруднит работу пользователей. Оптимальным является комбинированный подход: device control + специализированная защита HID + обучение персонала и жёсткий контроль физического доступа.

ЗАКЛЮЧЕНИЕ

В ходе выполнения данной научно-исследовательской работы была достигнута поставленная цель — выявить и проанализировать угрозы информационной безопасности, связанные с использованием микроконтроллеров с USB-интерфейсом в сценариях атак класса BadUSB, а также апробировать элементы двухуровневой методики исследования USB/HID-угроз.

В соответствии с целью работы были решены основные поставленные задачи. Изучены архитектура и возможности микроконтроллеров с USB-интерфейсом, используемых для эмуляции USB/HID-устройств. Проанализированы уязвимости протокола USB и интерфейса HID, показано, что современная модель доверия операционных систем к устройствам ввода не предусматривает криптографической аутентификации и не различает «человеческий» и автоматически сгенерированный ввод. На этой основе сформирована упрощённая модель угроз BadUSB и предложена прикладная классификация атак с использованием USB-микроконтроллеров по типу эмулируемого устройства и характеру воздействия.

В практической части был развёрнут экспериментальный Linux-стенд для анализа USB/HID-устройств. Получены реальные HID-дескрипторы мыши и клавиатуры, на их основе сформирован исходный корпус, а затем реализован прототип mutator, ориентированного на структуру дескрипторов, генерирующий модифицированные варианты HID-дескрипторов при сохранении общей структуры. Дополнительно были исследованы встроенные и policy-based средства защиты Linux, включая механизм авторизации USB-устройств и USBGuard, что позволило практически подтвердить возможность ограничения и блокировки нежелательных USB-подключений.

Отдельным результатом стало подтверждение аппаратного уровня методики на платформе RP2040 Zero. На базе TinyUSB были реализованы

базовый и модифицированный варианты HID-устройства, успешно перечисляемые системой. Это показало, что результаты с хоста могут быть перенесены на реальную микроконтроллерную платформу и использованы для воспроизведения USB/HID-сценариев на физическом устройстве.

Таким образом, выполненная работа подтвердила актуальность проблемы атак класса BadUSB и показала, что доступные микроконтроллерные платформы могут использоваться не только для демонстрации атакующего потенциала, но и как средство экспериментального исследования поведения USB/HID-стека и механизмов защиты. Практическая значимость результатов заключается в формировании исходного корпуса HID-дескрипторов, реализации прототипа mutator, проверке механизмов Linux authorization и USBGuard, а также в подтверждении возможности аппаратного воспроизведения USB/HID-сценариев на платформе RP2040 Zero.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Nohl K., Lell J. BadUSB - On Accessories that Turn Evil [Электронный ресурс] // Black Hat USA 2014. URL: https://www.blackhat.com/us-14/video/badusb-on-accessories-that-turn-evil.html (дата обращения: 20.03.2026).
2. Greenberg A. The Unpatchable Malware That Infects USBs Is Now on the Loose [Электронный ресурс] // Wired. 2014. URL: https://www.wired.com/2014/10/code-published-for-unfixable-usb-attack/ (дата обращения: 20.03.2026).
3. Roy S. What Is a BadUSB? Understand the Threat and How to Prevent It [Электронный ресурс] // Ivanti Blog. 2025. URL: https://www.ivanti.com/blog/what-is-badusb (дата обращения: 20.03.2026).
4. Building cheaper version of Rubber Ducky using Digispark-ATTiny85 [Электронный ресурс] // Tactical Adversary. URL: https://tacticaladversary.io/blog/build-cheaper-version-of-rubber-ducky-using-digispark-attiny85/ (дата обращения: 20.03.2026).
5. Detecting a Rubber Ducky USB Attack with Digital Guardian Advanced Threat Protection [Электронный ресурс] // Digital Guardian. 2016. URL: https://www.digitalguardian.com/index.php/blog/detecting-rubber-ducky-usb-attack-digital-guardian-advanced-threat-protection-video-demo (дата обращения: 20.03.2026).
6. Introduction to HID report descriptors [Электронный ресурс] // The Linux Kernel documentation. URL: https://docs.kernel.org/hid/hidintro.html (дата обращения: 20.03.2026).
7. Authorizing (or not) your USB devices to connect to the system [Электронный ресурс] // The Linux Kernel documentation. URL: https://docs.kernel.org/6.7/usb/authorization.html (дата обращения: 20.03.2026).

8. Peng H., Payer M. USBFuzz: A Framework for Fuzzing USB Drivers by Device Emulation [Электронный ресурс] // 29th USENIX Security Symposium (USENIX Security 20). 2020. P. 2559–2575. URL: https://www.usenix.org/conference/usenixsecurity20/presentation/peng (дата обращения: 20.03.2026).

9. Включение и выключение Защиты от атак BadUSB [Электронный ресурс] // Kaspersky Support. 2024. URL: https://support.kaspersky.ru/kes11/138434 (дата обращения: 20.03.2026).

10. Jang J., Kang M., Song D. ReUSB: Replay-Guided USB Driver Fuzzing [Электронный ресурс] // 32nd USENIX Security Symposium (USENIX Security 23). 2023. P. 2921–2938. URL: https://www.usenix.org/conference/usenixsecurity23/presentation/jang (дата обращения: 20.03.2026).

11. Rule Language [Электронный ресурс] // USBGuard Documentation. URL: https://usbguard.github.io/documentation/rule-language (дата обращения: 20.03.2026).

12. Configuration [Электронный ресурс] // USBGuard Documentation. URL: https://usbguard.github.io/documentation/configuration (дата обращения: 20.03.2026).

13. Fisk R. USG: The USG is Good, not Bad [Электронный ресурс] // GitHub. URL: https://github.com/robertfisk/USG (дата обращения: 20.03.2026).

14. Malicious Cable Detector by O.MG [Электронный ресурс] // Hak5. URL: https://shop.hak5.org/products/malicious-cable-detector-by-o-mg (дата обращения: 20.03.2026).

15. Loe E. L. SandUSB: An Installation-Free and User-Controllable Sandbox For USB Peripherals [Электронный ресурс]. 2016. URL: https://scholars.lib.ntu.edu.tw/entities/publication/aff1fc1a-aa08-4655-8e15-8cc5dc9749d3 (дата обращения: 20.03.2026).

16. Mohammadmoradi H., Gnawali O. Making Whitelisting-Based Defense Work Against BadUSB [Электронный ресурс] // Proceedings of the 2nd International Conference on Smart Digital Environment (ICSDE 2018). 2018. P. 127–134. URL: https://www2.cs.uh.edu/~gnawali/papers/badusb-icsde2018.pdf (дата обращения: 20.03.2026).